# Technical Documentation: UWB Communication (DWM3001CDK)

## 1. Hardware Configuration (Device Tree Overlay)

To establish communication between the nRF52833 microcontroller and the DW3000 UWB module, we had to redefine the pin mapping via a `.overlay` file. The default configuration of the `nrf52833dk` board does not match the electrical schematic of the DWM3001CDK shield.

**File:** `nrf52833dk_nrf52833.overlay`

### Key Modifications:

- **Disabling Conflicting Peripherals:** `spi3` and `uart0` were disabled to free up pins P0.08 and P0.29.
- **Pin Mapping :**
  - **SCK:** P0.03
  - **MOSI:** P0.08
  - **MISO:** P0.29 (with internal Pull-up enabled)
  - **Chip Select (CS):** Port 1, Pin 06 (`&gpio1 6`)
  - **IRQ (Interrupt):** Port 1, Pin 02 (`&gpio1 2`)
  - **Reset:** Port 0, Pin 24 (`&gpio0 24`)

Overlay code :

```
&pinctrl {
    spi1_default: spi1_default {
        group1 {
            psels = <NRF_PSEL(SPIM_SCK, 0, 3)>,
                    <NRF_PSEL(SPIM_MOSI, 0, 8)>,
                    <NRF_PSEL(SPIM_MISO, 0, 29)>;
            bias-pull-up; /* Safety pull-up on MISO */
        };
    };

    spi1_sleep: spi1_sleep {
        group1 {
            psels = <NRF_PSEL(SPIM_SCK, 0, 3)>,
                    <NRF_PSEL(SPIM_MOSI, 0, 8)>,
```

```
                    <NRF_PSEL(SPIM_MISO, 0, 29)>;
            low-power-enable;
        };
    };
};

/* Disable SPI3 and UART0 to avoid conflicts on P0.08 or P0.29 */
&spi3 { status = "disabled"; };
&uart0 { status = "disabled"; };

/* Enable SPI1 with the correct CS on Port 1 */
&spi1 {
    compatible = "nordic,nrf-spim";
    status = "okay";
    pinctrl-0 = <&spi1_default>;
    pinctrl-1 = <&spi1_sleep>;
    pinctrl-names = "default", "sleep";

    /* CS is on Port 1, Pin 06 */
    cs-gpios = <&gpio1 6 GPIO_ACTIVE_LOW>;

    dw3000@0 {
        compatible = "decawave,dw3000";
        status = "okay";
        reg = <0>;
        spi-max-frequency = <2000000>; /* 2 MHz for testing stability */

        /* IRQ is on Port 1, Pin 02 */
        irq-gpios = <&gpio1 2 GPIO_ACTIVE_HIGH>;

        /* Reset remains on P0.24 as per schematic */
        reset-gpios = <&gpio0 24 GPIO_ACTIVE_LOW>;
    };
};
```

## 2. Software and Timing Adjustments (DS-TWR)

### Context: Relaxing Constraints

We increased the delays to provide a larger time margin for the MCU to process interrupts and schedule radio events.

**Responder Modifications (ds_twr_responder.c)**
Delays were increased to allow the Responder enough time to switch from RX to TX.

| **Macro** | **Old Value** | **New Value** | **Function** |
|---|---|---|---|
| POLL_RX_TO_RESP_TX_DLY_UUS | 900 µs | **1500 µs** | Delay before sending the *Response* to the *Poll*. |
| RESP_TX_TO_FINAL_RX_DLY_UUS | 500 µs | **1200 µs** | Delay before enabling RX for the *Final* message. |
| FINAL_RX_TIMEOUT_UUS | 220 µs | **3000 µs** | RX window widened to ensure *Final* capture. |
| PRE_TIMEOUT | 5 | **0** | Preamble detection timeout disabled (infinite wait). |

**Initiator Modifications (ds_twr_initiator.c)**
The Initiator timings were adjusted to match the Responder's new schedule.

| **Macro** | **Old Value** | **New Value** | **Function** |
|---|---|---|---|
| POLL_TX_TO_RESP_RX_DLY_UUS | 300 µs + CPU_PROCESSING_TIME | **1200 µs** | Delay before enabling RX for the *Response*. |
| RESP_RX_TO_FINAL_TX_DLY_UUS | 300 µs + CPU_PROCESSING_TIME | **1500 µs** | Delay before sending the *Final* message. |

| | | | |
|---|---|---|---|
| RESP_RX_TIMEOUT_UUS | 300 µs | **3000 µs** | RX window widened to ensure *Response* capture. |
| PRE_TIMEOUT | 5 | **0** | Preamble detection timeout disabled. |

We replaced the dynamic calculation (CPU_PROCESSING_TIME) with a fixed safety margin to ensure the system always has enough time to process data before transmitting.

# 3. Build Configuration and Deployment

To achieve a functional Double-Sided Two-Way Ranging (DS-TWR) system, specific build steps must be followed.

## A. CMakeLists.txt & Source Selection

The `CMakeLists.txt` file must be modified to include the appropriate source file depending on the desired role. Since the project contains both codes, one must be selected for compilation:

- For Board A: Enable `ds_twr_initiator.c`
- For Board B: Enable `ds_twr_responder.c`

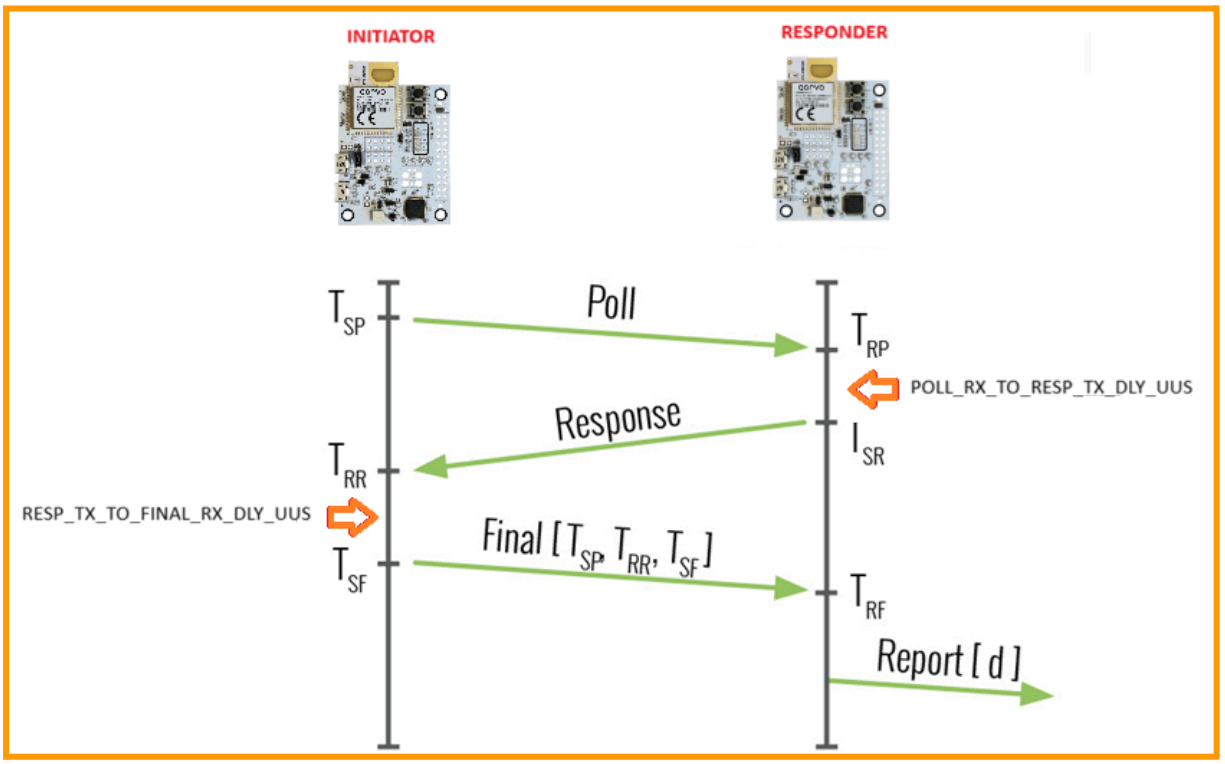
## B. Board Selection in nRF Connect

In **nRF Connect for VS Code**, the build configuration must target the correct board definition that includes our custom overlay:

- **Target Board:** `nrf52833dk/nrf52833`
- **Overlay:** Ensure `nrf52833dk_nrf52833.overlay` is detected and applied during the build process.

## C. Flashing Procedure

A complete ranging system requires two distinct physical devices:

1. **Device A (Initiator):** Flash the firmware compiled with the Initiator logic.
2. **Device B (Responder):** Flash the firmware compiled with the Responder logic.

Once both boards are powered, the ranging exchange initiates automatically, and the calculated distance is output via the RTT/Serial console on the Responder side.